TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 9: Theoretical Questions

Foreign Trade University

Instructions

Part A contains 16 questions requiring written explanations or short proofs. Part B is a 10-question quiz (true/false and multiple choice). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

Part A: Theory Questions

A1. State the four-step "recipe" for designing a dynamic-programming algorithm (subproblems, recurrence, evaluation order, reconstruction). Which step is usually the hardest, and why?

A2. State the Sequence Alignment problem precisely: what are the inputs (the two strings, the gap cost δ, and the mismatch costs α_{pq}), what is a valid alignment, and what is the objective? How does Levenshtein edit distance arise as a special case?

A3. Derive the alignment recurrence by reasoning about the *last column* of an optimal alignment of $x_1 \cdots x_i$ and $y_1 \cdots y_j$. List the three cases and state the base cases.

A4. Prove that the alignment recurrence is correct (i.e. that $\mathrm{OPT}(i, j)$ equals the true minimum-cost alignment of the two prefixes). Make clear where you use the fact that the rest of an optimal alignment, after fixing the last column, must itself be optimal.

A5. What are the time and space complexities of the basic table-filling alignment algorithm? Explain why the time is $O(mn)$ in terms of (number of subproblems) $\times$ (work per subproblem).

A6. Describe how to reconstruct an actual optimal alignment (the two padded strings with gap symbols) once the table is filled. Why does the traceback take only $O(m+n)$ time even though the table has $O(mn)$ cells?

A7. Explain, at a high level, Hirschberg's idea for computing an optimal alignment in only $O(m+n)$ space (while keeping $O(mn)$ time). What single observation about the dependency structure of the table makes the space saving possible, and what extra idea (divide-and-conquer) is needed to still recover the alignment, not just its cost?

A8. State the Longest Common Subsequence (LCS) recurrence and explain the reasoning behind each case (when $x_i = y_j$ versus $x_i \neq y_j$). In what sense is LCS a special case of the sequence-alignment framework?

A9. Prove the identity $d(X, Y) = m + n - 2\,\mathrm{LCS}(X, Y)$, where d is the minimum number of single-character insertions and deletions (no substitutions) needed to turn X (length m) into Y (length n).

A10. Explain why Dijkstra's algorithm can return incorrect answers on graphs with negative edge weights. Give (or describe) a small example where Dijkstra fails. Also explain why "shortest path" is ill-defined when a negative cycle is reachable.

A11. State the Bellman–Ford DP: define the subproblem $\mathrm{OPT}(k, v)$, write the recurrence (reasoning about the last edge), and state the base cases. Why is k bounded by $n - 1$ when there is no negative cycle?

A12. Prove that after $n-1$ passes of edge relaxation, Bellman–Ford computes correct shortest-path distances in a graph with no negative cycle. State the running time and justify it.

A13. Describe how Bellman–Ford detects a negative cycle, and how one can recover the actual cycle (not just a yes/no flag). Why does running exactly one additional (n-th) relaxation pass suffice as the test?

A14. State the Floyd–Warshall recurrence: define $d_k(i, j)$ in terms of allowed intermediate vertices, write the recurrence (reasoning about whether vertex k is used), and state the base case. What is the running time, and how does a negative cycle reveal itself in the final matrix?

A15. Explain why shortest (and even *longest*) paths in a DAG can be computed in $O(n + m)$ time by relaxing edges in topological order. Why is the longest-path problem tractable on a DAG but NP-hard on general graphs?

A16. State the Matrix Chain Multiplication recurrence (define $M(i, j)$, the split-point minimisation, and the base case) and the Grid Minimum-Path-Sum recurrence. For each, give the number of subproblems and the work per subproblem, and hence the total running time.

Part B: Quiz

B1. (Multiple choice) The sequence-alignment dynamic program runs in time:

(a) $O(m + n)$
(b) $O(mn)$
(c) $O(2^{m+n})$
(d) $O(mn \log(mn))$

B2. (True/False) In the alignment recurrence, the cell $\mathrm{OPT}(i, j)$ depends only on the three cells $\mathrm{OPT}(i-1, j-1)$, $\mathrm{OPT}(i-1, j)$, and $\mathrm{OPT}(i, j-1)$.

B3. (Short answer) The Levenshtein edit distance between `kitten` and `sitting` is what number, and what three single-character operations achieve it?

B4. (Multiple choice) For strings of lengths m and n, the relation between insert/delete-only edit distance d and LCS length L is:

(a) $d = m + n - L$
(b) $d = m + n - 2L$
(c) $d = \max(m, n) - L$
(d) $d = 2L - m - n$

B5. (True/False) Dijkstra's algorithm always computes correct shortest paths, even when some edge weights are negative, as long as there is no negative cycle.

B6. (Multiple choice) Bellman–Ford runs in time:

(a) $O(m \log n)$
(b) $O(m + n)$
(c) $O(mn)$
(d) $O(n^3)$

B7. (Short answer) After running Bellman–Ford's $n - 1$ relaxation passes, you run one more pass and find an edge (u, v, w) with $\mathrm{dist}[u] + w < \mathrm{dist}[v]$. What does this tell you about the graph?

B8. (Multiple choice) Floyd–Warshall computes all-pairs shortest paths in time:

(a) $O(n^2)$
(b) $O(n^3)$
(c) $O(mn)$
(d) $O(n^2 \log n)$

B9. (True/False) In a directed acyclic graph, both shortest *and* longest paths from a source can be computed in $O(n + m)$ time by relaxing edges in topological order.

B10. (Short answer) In matrix chain multiplication, $M(i, j)$ is computed by minimising over split points k with $i \le k < j$. Write the cost contributed by choosing split point k (in terms of M and the dimensions p_{i-1}, p_k, p_j).

Part C: Solutions

Part A

A1. The recipe: **(1) Define subproblems** – a small family of subproblems, each indexed by one or more parameters, whose optima combine into the whole. **(2) Write a recurrence** – express a subproblem's optimum in terms of smaller subproblems' optima, by reasoning about the *last decision* an optimal solution makes. **(3) Order the computation** – evaluate subproblems so each is solved before it is needed (e.g. smallest first, or in topological order). **(4) Reconstruct** – trace back (or store parent / choice pointers) to recover an actual optimal solution, not just its value. Step (2) is usually hardest: identifying the right notion of "last decision" and proving the rest of an optimal solution decomposes into independent, already-defined subproblems. Steps (1), (3), (4) are then largely mechanical, and the running time is (number of subproblems) $\times$ (work per subproblem).

A2. **Input:** two strings $X = x_1 \cdots x_m$, $Y = y_1 \cdots y_n$; a gap cost $\delta \geq 0$; and mismatch costs $\alpha_{pq} \geq 0$ with $\alpha_{pp} = 0$. A **valid alignment** writes the two strings one above the other with gap symbols inserted so both have equal length and no column is two gaps; each column is a match (equal characters, cost 0), a mismatch (cost $\alpha_{x_i y_j}$), or a character against a gap (cost δ). The **objective** is an alignment of minimum total column-cost sum. **Levenshtein edit distance** is the special case $\delta = 1$, $\alpha_{pq} = 1$ for $p \neq q$: it counts the minimum number of insertions, deletions, and substitutions to transform X into Y.

A3. Look at the last column of an optimal alignment of $x_1 \cdots x_i$ and $y_1 \cdots y_j$. Order is preserved, so exactly one of three things happens to x_i and y_j: *(a)* they are aligned together – cost $\alpha_{x_i y_j}$ plus an optimal alignment of $x_1 \cdots x_{i-1}$ and $y_1 \cdots y_{j-1}$; *(b)* x_i is aligned with a gap (deleted) – cost δ plus an optimal alignment of $x_1 \cdots x_{i-1}$ and $y_1 \cdots y_j$; *(c)* y_j is aligned with a gap (inserted) – cost δ plus an optimal alignment of $x_1 \cdots x_i$ and $y_1 \cdots y_{j-1}$. Hence

$$\mathrm{OPT}(i,j) = \min\big(\alpha_{x_i y_j} + \mathrm{OPT}(i-1,j-1),\ \delta + \mathrm{OPT}(i-1,j),\ \delta + \mathrm{OPT}(i,j-1)\big),$$

with base cases $\mathrm{OPT}(i,0) = i\delta$ and $\mathrm{OPT}(0,j) = j\delta$.

A4. By induction on $i+j$. *Base cases:* aligning a non-empty prefix with the empty string forces one gap per character, cost $i\delta$ (resp. $j\delta$); these are obviously optimal (and the only) alignments. *Inductive step* ($i, j \geq 1$): take any optimal alignment A of the two prefixes and look at its last column, which is one of the three cases. Suppose it aligns x_i with y_j. Removing that column leaves an alignment A' of $x_1 \cdots x_{i-1}$ and $y_1 \cdots y_{j-1}$, and $\mathrm{cost}(A) = \alpha_{x_i y_j} + \mathrm{cost}(A')$. *Here is the key step:* A' must be an *optimal* alignment of those shorter prefixes – otherwise replacing A' by a cheaper alignment and re-appending the last column would beat A, contradicting A's optimality. By the inductive hypothesis $\mathrm{cost}(A') = \mathrm{OPT}(i-1,j-1)$, so $\mathrm{cost}(A) = \alpha_{x_i y_j} + \mathrm{OPT}(i-1,j-1)$. The deletion and insertion cases are symmetric. Thus $\mathrm{cost}(A)$ equals one of the three expressions, so $\mathrm{OPT}(i,j) \geq \min(\cdots) = \mathrm{cost}(A)$ would be wrong – rather, $\mathrm{cost}(A) \geq \min(\cdots)$ since the min is over all three options. Conversely each of the three expressions is realised by an actual alignment (optimal alignment of the relevant smaller prefixes plus the matching last column), so the minimum is achievable. Hence $\mathrm{OPT}(i,j) = \mathrm{cost}(A) = \min(\cdots)$.

A5. There are $(m+1)(n+1) = O(mn)$ subproblems (cells). Each cell is computed by taking the minimum of three already-computed neighbouring cells plus $O(1)$ arithmetic, i.e. $O(1)$ work

per subproblem. Total time $= O(mn) \times O(1) = O(mn)$. The basic algorithm stores the whole table, so space is also $O(mn)$.

A6. Trace back from cell (m, n) toward $(0, 0)$. At each cell (i, j) determine which of the three terms achieved $\mathrm{OPT}(i, j)$: if the diagonal term, emit a column pairing x_i with y_j and move to $(i-1, j-1)$; if the "up" term, emit x_i against a gap and move to $(i-1, j)$; if the "left" term, emit a gap against y_j and move to $(i, j-1)$. Stop at $(0, 0)$ and reverse the emitted columns. Each step decreases $i + j$ by at least 1, so the traceback visits at most $m + n$ cells – hence $O(m+n)$ time – even though filling the table took $O(mn)$ (the traceback follows a single path through the table, not the whole grid).

A7. **Observation:** row i of the table depends only on row $i - 1$ (each cell uses the cell above, to the left, and diagonally up-left). So computing just the final *cost* $\mathrm{OPT}(m, n)$ needs only two rows in memory, i.e. $O(\min(m, n))$ space. But two rows do not retain enough information to trace the alignment back. **Hirschberg's extra idea:** divide and conquer. An optimal alignment must pass through some cell $(m/2, q)$ in the middle row. Compute, in linear space, the cost of optimally aligning the top half of X with each prefix $y_1 \cdots y_q$ (a forward sweep) and the cost of aligning the bottom half of X with each suffix $y_{q+1} \cdots y_n$ (a backward sweep); the optimal q minimises the sum. Recurse on the two independent halves. The work satisfies $T(m, n) = O(mn) + T(m/2, q) + T(m/2, n - q)$, which solves to $O(mn)$ time, while the space is $O(m + n)$.

A8. LCS recurrence: $L(i, j) = 0$ if $i = 0$ or $j = 0$; $L(i, j) = 1 + L(i - 1, j - 1)$ if $x_i = y_j$; and $L(i, j) = \max(L(i-1, j), L(i, j-1))$ if $x_i \neq y_j$. *Reasoning:* if $x_i = y_j$ there is always an optimal LCS that matches them (matching equal last characters never hurts), reducing to the LCS of the two shorter prefixes plus one. If $x_i \neq y_j$, they cannot *both* be the last matched character, so at least one is unused; dropping x_i gives $L(i - 1, j)$ and dropping y_j gives $L(i, j - 1)$, and we take the better. LCS is the special case of alignment that *maximises* score with match reward $+1$, mismatch 0, and gap 0 – the score of an alignment is then exactly the number of matched (equal) columns, i.e. the length of a common subsequence.

A9. ($\leq$) Let S be a longest common subsequence, $|S| = L$. Build an edit script that keeps the L characters of S in place: delete the $m - L$ characters of X not in S, then insert the $n - L$ characters of Y not in S. This transforms X into Y using $(m - L) + (n - L) = m + n - 2L$ insert/delete operations, so $d(X, Y) \leq m + n - 2L$. ($\geq$) Take any optimal insert/delete script turning X into Y. The characters of X that are *never deleted* survive into Y in their original relative order, so they form a common subsequence; say there are ℓ of them, with $\ell \leq L$. The script deletes $m - \ell$ characters of X and inserts $n - \ell$ characters to build Y, for $m + n - 2\ell \geq m + n - 2L$ operations. Hence $d(X, Y) \geq m + n - 2L$. Combining, $d(X, Y) = m + n - 2L$.

A10. Dijkstra finalises a vertex's distance the moment it is extracted from the priority queue (the minimum-tentative-distance unvisited vertex), assuming no later edge can reduce it – valid only with non-negative weights. With a negative edge, a vertex finalised early might later be reached more cheaply via a path through a negative edge, and Dijkstra never revisits it. *Example:* vertices s, a, b with edges $s \to a$ (weight 1), $s \to b$ (weight 4), $b \to a$ (weight -3). Dijkstra finalises a at distance 1 (extracted before b), but the true shortest distance to a is $4 + (-3) = 1$ here – to make it fail use $s \to a = 2$, $s \to b = 1$, $b \to a = -2$: Dijkstra finalises $a = 2$, but $s \to b \to a = 1 - 2 = -1$ is shorter. A **negative cycle** reachable en route makes

"shortest path" ill-defined: looping the cycle repeatedly lowers the cost without bound, so the infimum is $-\infty$ and no finite shortest path exists.

A11. **Subproblem:** $\mathrm{OPT}(k, v)$ = minimum cost of an s–v path using at most k edges. **Recurrence** (last edge): either an optimal $\le k$-edge path uses $\le k-1$ edges, or its last edge is some (u, v) preceded by an optimal $\le k-1$-edge path to u:

$$\mathrm{OPT}(k, v) = \min\Big(\mathrm{OPT}(k-1, v),\ \min_{(u,v)\in E}\big(\mathrm{OPT}(k-1, u) + w(u, v)\big)\Big),$$

with $\mathrm{OPT}(0, s) = 0$ and $\mathrm{OPT}(0, v) = \infty$ for $v \neq s$. **Bound on k:** if there is no negative cycle, every shortest path is *simple* (repeating a vertex would mean a cycle, and a non-negative cycle can be removed without increasing cost), so it has at most $n-1$ edges; thus $\mathrm{OPT}(n-1, v)$ already gives the true distances.

A12. By induction on the pass number k, the invariant "after pass k, $\mathrm{dist}[v] \le \mathrm{OPT}(k, v)$ for all v". *Base:* before any pass, $\mathrm{dist}[s] = 0 = \mathrm{OPT}(0, s)$, all others ∞. *Step:* assume it holds after pass $k-1$. Fix v and an optimal $\le k$-edge path P. If P uses $\le k-1$ edges, $\mathrm{dist}[v]$ is already $\le \mathrm{OPT}(k-1, v) = \mathrm{cost}(P)$. Otherwise P's last edge (u, v) follows a $\le k-1$-edge prefix of cost $\mathrm{OPT}(k-1, u)$; by hypothesis $\mathrm{dist}[u] \le \mathrm{OPT}(k-1, u)$ entering pass k, and relaxing (u, v) in pass k sets $\mathrm{dist}[v] \le \mathrm{dist}[u] + w \le \mathrm{cost}(P)$. With no negative cycle, shortest paths are simple ($\le n-1$ edges), so after $n-1$ passes $\mathrm{dist}[v]$ equals the true distance. *Running time:* each pass relaxes all m edges and there are $n-1$ passes, so $O(mn)$.

A13. **Detection:** after the $n-1$ passes, run one more pass; if any edge (u, v, w) still satisfies $\mathrm{dist}[u] + w < \mathrm{dist}[v]$, report a negative cycle. **Why one pass suffices:** in a negative-cycle-free graph, distances have converged after $n-1$ passes (previous question), so *no* edge can be further relaxed; therefore a successful relaxation on the n-th pass means distances are still decreasing, which can only happen if a reachable negative cycle lets paths grow beyond $n-1$ edges while still improving. **Recovering the cycle:** when the n-th-pass relaxation of (u, v) succeeds, set $\mathrm{parent}[v] = u$, then start at v and follow parent pointers n times (this guarantees ending at a vertex *on* the cycle, since the parent chain must eventually enter the cycle); call that vertex x, then follow parents from x until x recurs – the vertices traversed form the negative cycle.

A14. **Subproblem:** $d_k(i, j)$ = length of the shortest i–j path whose *intermediate* vertices all lie in $\{1, \ldots, k\}$. **Recurrence** (is vertex k used?):

$$d_k(i, j) = \min\big(d_{k-1}(i, j),\ d_{k-1}(i, k) + d_{k-1}(k, j)\big),$$

with $d_0(i, j) = w(i, j)$ for edges, $d_0(i, i) = 0$, else ∞. Either the optimal path avoids k (first term), or it passes through k exactly once, splitting into an $i \to k$ and a $k \to j$ path each using only $\{1, \ldots, k-1\}$ as intermediates (second term). With n values of k and n^2 pairs (i, j), each $O(1)$, the time is $O(n^3)$ and space $O(n^2)$ (one matrix updated in place). A **negative cycle** shows up as some diagonal entry $d_n(i, i) < 0$ (a path from i back to i of negative length).

A15. A DAG has a **topological order** in which every edge goes from an earlier to a later vertex. Processing vertices in this order, when we reach v all of its predecessors u (with $(u, v) \in E$) are already finalised, so $\mathrm{dist}(v) = \min_{(u,v)\in E}(\mathrm{dist}(u) + w)$ (or max for longest paths) is computed in one sweep. Each edge is examined exactly once, giving $O(n + m)$ (plus $O(n + m)$ for the topological sort). Longest paths are tractable on a DAG precisely because there are no cycles

– the recurrence is well-founded and longest *walks* equal longest *paths*. On a general graph, a longest *simple* path is NP-hard (e.g. a Hamiltonian path is a longest simple path of $n-1$ edges), and longest walks are unbounded if any positive cycle exists, so no analogous DP works.

A16. Matrix chain: $M(i,j)$ = minimum scalar multiplications to compute $A_i \cdots A_j$, with A_t of dimension $p_{t-1} \times p_t$:

$$M(i,j) = \min_{i \le k < j} \big(M(i,k) + M(k+1,j) + p_{i-1}p_k p_j\big), \qquad M(i,i) = 0.$$

There are $O(n^2)$ subproblems (i,j), each minimising over $O(n)$ split points, so $O(n^3)$ time. **Grid minimum path sum:** with cell costs $c(i,j)$ and moves right/down,

$$S(i,j) = c(i,j) + \min\big(S(i-1,j),\ S(i,j-1)\big),$$

($S(0,0) = c(0,0)$, first row/column accumulate). There are $O(rc)$ subproblems, each $O(1)$ work, so $O(rc)$ time.

Part B

B1. (b) $O(mn)$. There are $(m+1)(n+1)$ cells, each computed in $O(1)$.

B2. True. Each alignment cell uses the cell diagonally up-left (match/mismatch), directly above (deletion), and directly left (insertion).

B3. Edit distance **3**. One optimal sequence: substitute `k`→`s` (`kitten`→`sitten`), substitute `e`→`i` (`sitten`→`sittin`), insert `g` at the end (`sittin`→`sitting`).

B4. (b) $d = m+n-2L$. Each unmatched character of X is deleted and each unmatched character of Y is inserted; L characters are shared.

B5. False. Dijkstra can fail with negative edges even without a negative cycle, because it finalises distances assuming no later (negative) edge can improve them. Use Bellman–Ford instead.

B6. (c) $O(mn)$. $n-1$ passes, each relaxing all m edges.

B7. It tells you the graph contains a **negative cycle** reachable from the source (distances should have converged after $n-1$ passes if none existed).

B8. (b) $O(n^3)$. Three nested loops over k, i, j, each 1 to n.

B9. True. A topological order lets a single $O(n+m)$ sweep compute shortest paths (with min) or longest paths (with max); acyclicity makes longest paths well-defined.

B10. The cost contributed by split point k is $M(i,k) + M(k+1,j) + p_{i-1}\,p_k\,p_j$ (cost of the left block, the right block, and the final multiply of the two resulting matrices).